

# CAP 4611 Assignment 2

Jayden Rodriguez Lugo - 5505335 and David Navarrete - 5479357

## Important: Submission Format [5 points]

Please make s sure to follow the submission instructions posted on the course website and on the webcourses. Here is a direct link to the instructions. [We will deduct marks if the submission format is incorrect, or if your handwriting is at all difficult to read](#) – at least these 5 points, more for egregious issues. Be sure to include your name and student number. If you are working in a group of two, include both names and student numbers and only upload one submission. For more detailed instructions, see the link above.

## 1 K-Nearest Neighbours [15 points]

*Assistance: OpenAI Codex assisted with code corrections, explanations, and verification; numerical results were computed from the supplied datasets.*

In the *citiesSmall* dataset (introduced in A1), nearby points tend to receive the same class label because they are part of the same U.S. state. For this problem, perhaps a  $k$ -nearest neighbours classifier might be a better choice than a decision tree. The file *knn.py* has implemented the training function for a  $k$ -nearest neighbour classifier (which is to just memorize the data).

Fill in the `predict` function in *knn.py* so that the model file implements the  $k$ -nearest neighbour prediction rule. You should use Euclidean distance, and may find numpy's `sort` and/or `argsort` functions useful. You can also use `utils.euclidean_dist_squared`, which computes the squared Euclidean distances between all pairs of points in two matrices. Also, please note that you will have to write code in the *main.py* file in order to do parts 2 and 3 of this question.

1. Write the `predict` function. Note that you can use the `utils.mode()` function in your implementation, and you can rely on this function to correctly handle potential ties when assigning a label to a training data point. [Submit this code.](#) [5 points]

Answer:

```
def predict(self, X_hat):
    #fills in the k-nearest neighbor prediction rule
    #distance = num of training points(i) * num of test points(j)
    distance = euclidean_dist_squared(self.X, X_hat)

    #array to hold the predictions for each test case
    Y_hat = np.zeros(X_hat.shape[0])

    #goes through the test points
    for i in range(X_hat.shape[0]):
        #gets and sorts the current distance
        cur_dist = distance[:, i]
        sorted_indices = np.argsort(cur_dist)
```

```

        #Gets first k indicies, gets label of k nearest training points, Predict most common
        label
        Y_hat[i] = utils.mode(self.y[sorted_indices[:self.k]])

    #return the predictions
    return Y_hat

```

2. Report the training and test error obtained on the *citiesSmall* dataset for  $k = 1$ ,  $k = 3$ , and  $k = 10$ . *Optionally*, try running a decision tree on this same train/test split; which gets better test accuracy? [4 points]

Answer:

| $k$ | Training error | Test error |
|-----|----------------|------------|
| 1   | 0.0000         | 0.0645     |
| 3   | 0.0275         | 0.0660     |
| 10  | 0.0575         | 0.0850     |

Among these three choices,  $k = 1$  has the lowest test error (6.45%).

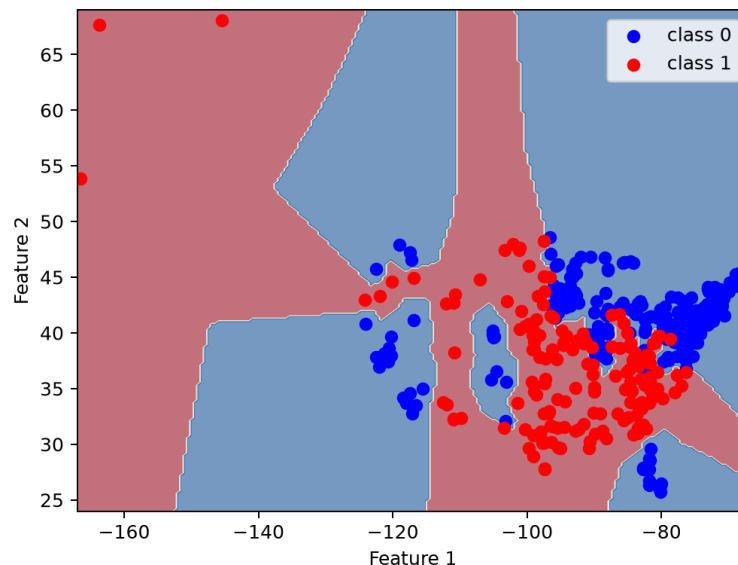
```

for k in [1, 3, 10]:
    model = KNN(k)
    model.fit(X, y)
    print(k, np.mean(model.predict(X) != y),
          np.mean(model.predict(X_test) != y_test))
model = KNN(1)
model.fit(X, y)
utils.plot_classifier(model, X, y)
plt.savefig('../figs/q1_knn_boundary.png')

```

3. Generate a plot with `utils.plot_classifier` on the *citiesSmall* dataset (plotting the training points) for  $k = 1$ , using your implementation of kNN. Include the plot here. To see if your implementation makes sense, you might want to check against the plot using `sklearn.neighbors.KNeighborsClassifier`. Remember that the assignment 1 code had examples of plotting with this function and saving the result, if that would be helpful. [2 points]

Answer:



4. Why is the training error 0 for  $k = 1$ ? [2 points]

Answer: For  $k = 1$ , a training example is its own nearest neighbour at distance zero, so it receives its own label. Thus the training error is zero on this dataset, with ties handled as specified.

5. Recall that we want to choose hyper-parameters so that the test error is (hopefully) minimized. How would you choose  $k$ ? [2 points]

Answer: Choose the  $k$  with the lowest mean validation error in cross-validation performed only on the training data. Then refit on the entire training set and use the held-out test set once to estimate generalization error.

## 2 Picking $k$ in kNN [15 points]

Assistance: OpenAI Codex assisted with code corrections, explanations, and verification; numerical results were computed from the supplied datasets. The file `data/ccdata.pkl` contains a subset of Statistics Canada's 2019 Survey of Financial Security; we're predicting whether a family regularly carries credit card debt, based on a bunch of demographic and financial information about them. (You might imagine social science researchers wanting to do something like this if they don't have debt information available – or various companies wanting to do it for less altruistic reasons.) If you're curious what the features are, you can look at the `'feat_descs'` entry in the dataset dictionary.

Anyway, now that we have our kNN algorithm working,<sup>1</sup> let's try choosing  $k$  on this data!

1. Remember the golden rule: we don't want to look at the test data when we're picking  $k$ . Inside the `q2()` function of `main.py`, implement 10-fold cross-validation, evaluating on the `ks` set there (1, 5, 9, ..., 29), and store the *mean* accuracy across folds for each  $k$  into a variable named `cv_accs`.

Specifically, make sure you test on the first 10% of the data after training on the remaining 90%, then test on 10% to 20% and train on the remainder, etc – don't shuffle (so your results are consistent with ours; the data is already in random order). Implement this yourself, don't use scikit-learn or any other existing implementation of splitting. There are lots of ways you could do this, but one reasonably convenient way is to create a numpy "mask" array, maybe using `np.ones(n, dtype=bool)` for an all-True array of length `n`, and then setting the relevant entries to `False`. It also might be helpful to know that `~ary` flips a boolean array (True to False and vice-versa).

[Submit this code](#), following the general submission instructions to include your code in your results file. [5 points]

Answer:

```
def q2():
    dataset = load_dataset("ccdebt.pkl")
    X = dataset["X"]
    y = dataset["y"]
    X_test = dataset["Xtest"]
    y_test = dataset["ytest"]

    ks = list(range(1, 30, 4))
    cv_accs = [] #stores the mean accuracy across folds for each k
    cv_err = [] #stores the mean error across folds for each k
    num_train_ex = X.shape[0]
    fold_size = num_train_ex // 10
```

---

<sup>1</sup>If you haven't finished the code for question 1, or if you'd just prefer a slightly faster implementation, you can use scikit-learn's `KNeighborsClassifier` instead. The `fit` and `predict` methods are the same; the only difference for our purposes is that `KNN(k=3)` becomes `KNeighborsClassifier(n_neighbors=3)`.

```

#implement 10-fold cross-validation
for depth in ks:
    #stores the current fold
    fold_accs = []
    fold_err = []
    #preforms the 10-fold cross validation
    for fold in range(10):
        #begin with everything marked as true
        mask = np.ones(num_train_ex, dtype=bool)
        start = fold * num_train_ex // 10
        end = (fold + 1) * num_train_ex // 10

        #mark validation examples as false
        mask[start:end] = False

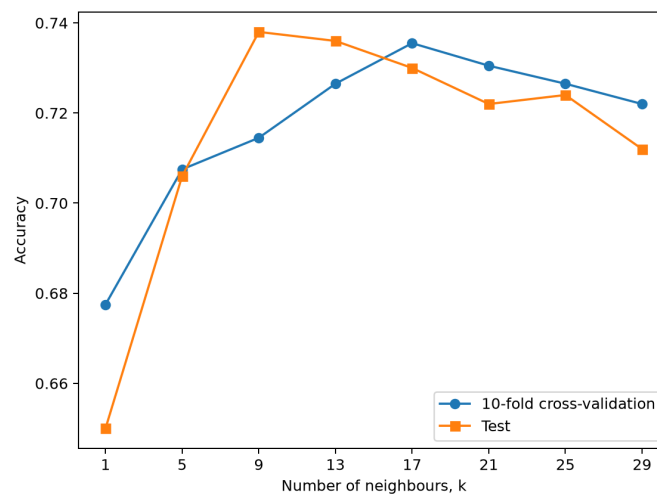
        #train KNN
        model = KNN(depth)
        model.fit(X[mask], y[mask])

        #make predictions, get the accuracy
        y_predictions = model.predict(X[~mask])
        accuracy = np.mean(y_predictions == y[~mask])
        foldError = np.mean(y_predictions != y[~mask])
        #inputs current accuracy into the array
        fold_accs.append(accuracy)
        fold_err.append(foldError)
    #input folds array into the cross validation array
    cv_accs.append(np.mean(fold_accs))
    cv_err.append(np.mean(fold_err))

```

2. The point of cross-validation is to get a sense of what the test error for a particular value of  $k$  would be. Implement, similarly to the code you wrote for question 1.2, a loop to compute the test accuracy for each value of  $k$  above. [Submit a plot the cross-validation and test accuracies as a function of  \$k\$ .](#) Make sure your plot has axis labels and a legend. [5 points]

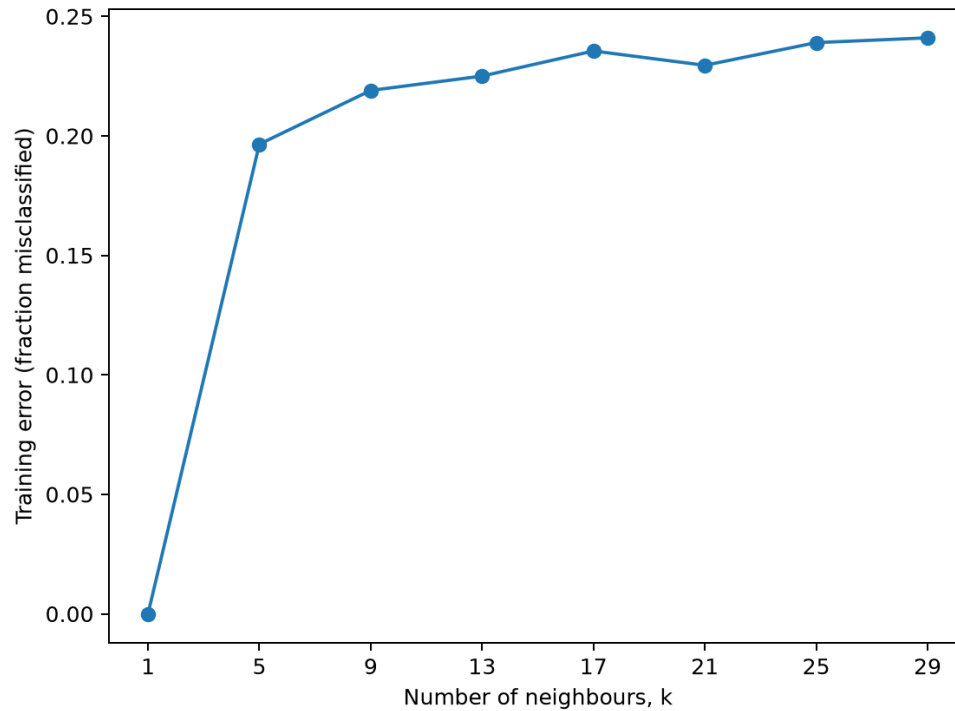
Answer:



3. Which  $k$  would cross-validation choose in this case? Which  $k$  has the best test accuracy? Would the cross-validation  $k$  do okay (qualitatively) in terms of test accuracy? [2 points]

Answer: Cross-validation chooses  $k = 17$ , with mean accuracy 0.7355. Test accuracy is highest at  $k = 9$  (0.7380). The cross-validation choice performs well: its test accuracy is 0.7300, only 0.8 percentage points below the best observed test accuracy. The test comparison is retrospective; it is not used to choose  $k$ .

4. Separately, submit a plot of the training error as a function of  $k$ . How would the  $k$  with best training error do in terms of test error, qualitatively? [3 points]



Answer: The  $k$  with the best training error would be  $k = 1$  (lowest training error), but it would typically do poorly on test error because it overfits (high variance).

### 3 Naïve Bayes [17 points]

Assistance: OpenAI Codex assisted with code corrections, explanations, and verification; numerical results were computed from the supplied datasets.

In this section we'll implement Naïve Bayes, a very fast classification method that is often surprisingly accurate for text data with simple representations like bag of words.

#### 3.1 Naïve Bayes by Hand [5 points]

Consider the dataset below, which has 10 training examples and 3 features:

$$X = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad y = \begin{bmatrix} \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{spam} \\ \text{not spam} \\ \text{not spam} \\ \text{not spam} \\ \text{not spam} \end{bmatrix}.$$

The feature in the first column is <your name> (whether the e-mail contained your name), in the second column is “lottery” (whether the e-mail contained this word), and the third column is “Venmo” (whether the e-mail contained this word). Suppose you believe that a naive Bayes model would be appropriate for this dataset, and you want to classify the following test example:

$$\hat{x} = [1 \quad 1 \quad 0].$$

##### 3.1.1 Prior probabilities [1 points]

Compute the estimates of the class prior probabilities, which I also called the “baseline spam-ness” in class. (you don't need to show any work):

- $\Pr(\text{spam})$ .

Answer: 6/10

- $\Pr(\text{not spam})$ .

Answer: 4/10

##### 3.1.2 Conditional probabilities [1 points]

Compute the estimates of the 6 conditional probabilities required by Naïve Bayes for this example (you don't need to show any work):

- $\Pr(\text{<your name>} = 1 \mid \text{spam})$ .

Answer: 1/6

- $\Pr(\text{lottery} = 1 \mid \text{spam})$ .

Answer: 5/6

- $\Pr(\text{Venmo} = 0 \mid \text{spam})$ .

Answer: 1/3 or 2/6

- $\Pr(\text{<your name>} = 1 \mid \text{not spam})$ .

Answer:  $3/4$

- $\Pr(\text{lottery} = 1 \mid \text{not spam})$ .

Answer:  $1/4$

- $\Pr(\text{Venmo} = 0 \mid \text{not spam})$ .

Answer:  $3/4$

### 3.1.3 Prediction [2 points]

Under the naive Bayes model and your estimates of the above probabilities, what is the most likely label for the test example? (Show your work.)

Answer:  $\Pr(\text{Spam}) = .6$ , scores  $(1/6, 5/6, 2/6)$ ,  $\Pr(\text{not spam}) = .4$ , scores  $(3/4, 1/4, 3/4)$ .  $.6(1/6)(5/6)(2/6) = 1/36$ ,  $.4(3/4)(1/4)(3/4) = 9/160$ .  $1/36 = .028$ ,  $9/160 = .056$ .  $1/36 < 9/160$

most likely to be [NOT SPAM]

### 3.1.4 Simulating Laplace Smoothing with Data [1 points]

One way to think of Laplace smoothing is that you're augmenting the training set with extra counts. Consider the estimates of the conditional probabilities in this dataset when we use Laplace smoothing (with  $\beta = 1$ ). Give a set of extra training examples where, if they were included in the training set, the "plain" estimation method (with no Laplace smoothing) would give the same estimates of the conditional probabilities as using the original dataset with Laplace smoothing. Present your answer in a reasonably easy-to-read format, for example the same format as the data set at the start of this question.

Answer:

$$X = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} \text{spam} \\ \text{spam} \\ \text{not spam} \\ \text{not spam} \end{bmatrix}.$$

For each class these two additional examples add one zero and one one to every feature. For example,  $P(\text{name} = 1 \mid \text{spam}) = (1 + 1)/(6 + 2) = 2/8$ , exactly the estimate with  $\beta = 1$ .

## 3.2 Exploring Bag-of-Words [2 points]

If you run `python main.py 3.2`, it will load the following dataset:

1. **X**: A binary matrix. Each row corresponds to a newsgroup post, and each column corresponds to whether a particular word was used in the post. A value of 1 means that the word occurred in the post.
2. **wordlist**: The set of words that correspond to each column.
3. **y**: A vector with values 0 through 3, with the value corresponding to the newsgroup that the post came from.
4. **groupnames**: The names of the four newsgroups.
5. **Xvalidate** and **yvalidate**: the word lists and newsgroup labels for additional newsgroup posts.

Answer the following:

1. Which word corresponds to column 73 of *X*? (This is index 72 in Python.)

Answer: question

2. Which words are present in training example 803 (Python index 802)?

Answer: case, children, health, help, problem, program

3. Which newsgroup name does training example 803 come from?

Answer: talk.\*

### 3.3 Naïve Bayes Implementation [4 points]

If you run `python main.py 3.3` it will load the newsgroups dataset, fit a basic naive Bayes model and report the validation error.

The `predict()` function of the naive Bayes classifier is already implemented. However, in `fit()` the calculation of the variable `p_xy` is incorrect (right now, it just sets all values to  $1/2$ ). [Modify this function so that `p\_xy` correctly computes the conditional probabilities of these values based on the frequencies in the data set. Submit your code. Report the training and validation errors that you obtain.](#)

Answer:

```
p_xy = np.zeros((d, k))
for c in range(k):
    X_c = X[y == c]
    p_xy[:, c] = np.mean(X_c, axis=0)
```

Answer: Training error: 0.199852 (19.9852%). Validation error: 0.187662 (18.7662%). These results use the supplied prediction method and the corrected class-conditional estimates.

### 3.4 Laplace Smoothing Implementation [4 points]

Laplace smoothing is one way to prevent failure cases of Naïve Bayes based on counting. Recall what you know from lecture to implement Laplace smoothing to your Naïve Bayes model.

- Modify the `NaiveBayesLaplace` class provided in `naive_bayes.py` and write its `fit()` method to implement Laplace smoothing. [Submit this code.](#)

Answer:

```
def fit(self, X, y):
    n, d = X.shape
    self.p_y = np.bincount(y, minlength=self.num_classes) / n
    self.p_xy = np.zeros((d, self.num_classes))
    for c in range(self.num_classes):
        X_c = X[y == c]
        self.p_xy[:, c] = (X_c.sum(axis=0) + self.beta) / (
            X_c.shape[0] + 2 * self.beta
        )
```

- Using the same data as the previous section, fit Naïve Bayes models with **and** without Laplace smoothing to the training data. Use  $\beta = 1$  for Laplace smoothing. For each model, look at  $p(x_{ij} = 1 \mid y_i = 0)$  across all  $j$  values (i.e. all features) in both models. [Do you notice any difference? Explain.](#)

Answer: For class 0, the unsmoothed probabilities are  $N_{j,0}/N_0$ ; with  $\beta = 1$  they become  $(N_{j,0}+1)/(N_0+2)$ . Smoothing moves estimates toward  $1/2$ , making zero probabilities positive and probabilities of one smaller than one. This prevents one previously unseen feature value from forcing an entire class likelihood to zero. In the data, 12 class-0 probabilities are zero without smoothing and none are zero with  $\beta = 1$ ; the minimum increases from 0 to 0.000436872.



```

for beta in [0, 1, 10000]:
    model = NaiveBayes(4) if beta == 0 else NaiveBayesLaplace(4, beta)
    model.fit(X, y)
    print(beta, model.p_xy[:, 0])

```

- One more time, fit a Naïve Bayes model with Laplace smoothing using  $\beta = 10000$ . Look at  $p(x_{ij} = 1 \mid y_i = 0)$ . Do these numbers look like what you expect? Explain.

Answer: With  $\beta = 10000$ , the estimates  $(N_{j,0} + 10000)/(N_0 + 20000)$  are close to  $1/2$  for every feature. This is expected because the balanced pseudo-counts dominate the observed counts; features consequently become much less informative about the class. The observed class-0 probabilities range from 0.448692 to 0.473774, compared with 0 to 0.244425 without smoothing.

### 3.5 Runtime of Naïve Bayes for Discrete Data [2 points]

For a given training example  $i$ , the predict function in the provided code computes the quantity

$$p(y_i \mid x_i) \propto p(y_i) \prod_{j=1}^d p(x_{ij} \mid y_i),$$

for each class  $y_i$  (and where the proportionality constant is not relevant). For many problems, a lot of the  $p(x_{ij} \mid y_i)$  values may be very small. This can cause the above product to underflow. The standard fix for this is to compute the logarithm of this quantity and use that  $\log(ab) = \log(a) + \log(b)$ ,

$$\log p(y_i \mid x_i) = \log p(y_i) + \sum_{j=1}^d \log p(x_{ij} \mid y_i) + (\log \text{ of the irrelevant proportionality constant}).$$

This turns the multiplications into additions and thus typically would not underflow.

Assume you have the following setup:

- The training set has  $n$  objects each with  $d$  features.
- The test set has  $t$  objects with  $d$  features.
- Each feature can have up to  $c$  discrete values (you can assume  $c \leq n$ ).
- There are  $k$  class labels (you can assume  $k \leq n$ ).

You can implement the training phase of a naive Bayes classifier in this setup in  $O(nd)$ , since you only need to do a constant amount of work for each  $x_{ij}$  value. (You do not have to actually implement it in this way for the previous question, but you should think about how this could be done.) What is the cost of classifying  $t$  test examples with the model and this way of computing the predictions?

Answer: The cost is  $O(tkd)$ : for each of  $t$  examples we compute  $k$  class scores, each requiring  $d$  feature-probability lookups and log additions, followed by an  $O(k)$  maximum.

## 4 Random Forests [15 points]

Assistance: OpenAI Codex assisted with code corrections, explanations, and verification; numerical results were computed from the supplied datasets.

The file `vowels.pkl` contains a supervised learning dataset where we are trying to predict which of the 11 “steady-state” English vowels that a speaker is trying to pronounce.

You are provided with a `RandomStump` class that differs from `DecisionStumpInfoGain` in that it only considers  $\lfloor \sqrt{d} \rfloor$  randomly-chosen features.<sup>2</sup> You are also provided with a `RandomTree` class that is exactly the same as `DecisionTree` except that it uses `RandomStump` instead of `DecisionStump` and it takes a bootstrap sample of the data before fitting. In other words, `RandomTree` is the entity we discussed in class, which makes up a random forest.

If you run `python main.py 4` it will fit a deep `DecisionTree` using the information gain splitting criterion. You will notice that the model overfits badly.

1. Using the provided code, evaluate the `RandomTree` model of unlimited depth. Why doesn’t the random tree model have a training error of 0? [2 points]

Answer: A bootstrap sample omits about  $(1 - 1/n)^n \approx e^{-1} \approx 36.8\%$  of the original examples. A deep random tree may fit its sampled examples but misclassify omitted examples when evaluated on the full training set. The measured errors are reported in part 4 below.

2. For `RandomTree`, if you set the `max_depth` value to `np.inf`, why do the training functions terminate instead of making an infinite number of splitting rules? [2 points]

Answer: The recursion stops at a pure leaf or when no positive-information-gain split exists. Every accepted split produces two nonempty, strictly smaller subsets, so a finite dataset cannot support an infinite chain of splits, even without a depth limit.

3. Complete the `RandomForest` class in `random_tree.py`. This class takes in hyperparameters `num_trees` and `max_depth` and fits `num_trees` random trees each with maximum depth `max_depth`. For prediction, have all trees predict and then take the mode. Submit this code. [5 points]

Answer:

```
class RandomForest:
    def __init__(self, num_trees, max_depth):
        self.num_trees = num_trees
        self.max_depth = max_depth
        self.trees = []

    def fit(self, X, y):
        self.trees = []
        for _ in range(self.num_trees):
            tree = RandomTree(max_depth=self.max_depth)
            tree.fit(X, y)
            self.trees.append(tree)

    def predict(self, X_pred):
        predictions = np.array([tree.predict(X_pred) for tree in self.trees])
        return np.array([utils.mode(predictions[:, i])
                          for i in range(X_pred.shape[0])])
```

---

<sup>2</sup>The notation  $\lfloor x \rfloor$  means the “floor” of  $x$ , or “ $x$  rounded down”. You can compute this with `np.floor(x)` or `math.floor(x)`.

4. Using 50 trees, and a max depth of  $\infty$ , report the training and testing error. Compare this to what we got with a single `DecisionTree` and with a single `RandomTree`. Are the results what you expected? Discuss. [3 points]

Answer: With maximum depth  $\infty$ , the observed errors were:

| Model                    | Training error | Test error |
|--------------------------|----------------|------------|
| Decision tree            | 0.000          | 0.367      |
| Random tree              | 0.167          | 0.436      |
| Random forest (50 trees) | 0.000          | 0.159      |

The random tree and forest each use seed 4611, reset before fitting. The forest has much lower test error than either single tree while retaining zero training error. This matches the expected reduction in variance from voting across diverse trees; a single bootstrap tree can misclassify omitted training points and has unstable test predictions.

```
np.random.seed(4611)
evaluate_model(RandomTree(max_depth=np.inf))
np.random.seed(4611)
evaluate_model(RandomForest(num_trees=50, max_depth=np.inf))
# evaluate_model is the supplied training/test evaluation helper.
```

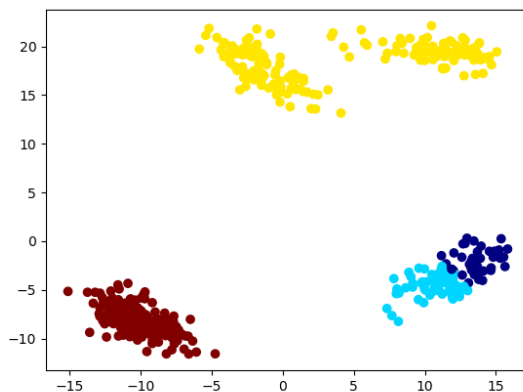
5. Why does a random forest typically have a training error of 0, even though random trees typically have a training error greater than 0? [3 points]

Answer: Each training example appears in about  $1 - e^{-1} \approx 63.2\%$  of the bootstrap samples. Deep trees that include it tend to predict its label correctly, so these votes usually form a majority; voting can also correct errors from trees that omitted it. Thus the forest often has zero training error, although this is not guaranteed for finite forests, restricted trees, or conflicting labels.

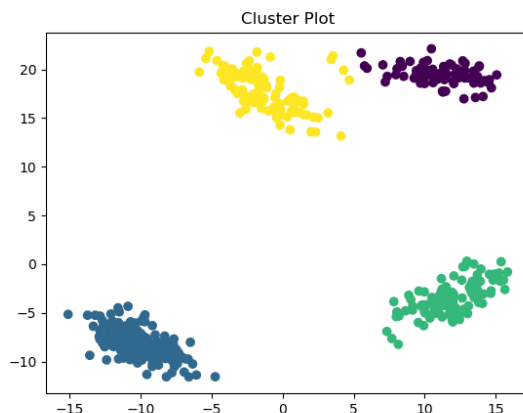
## 5 Clustering [15 points]

Assistance: OpenAI Codex assisted with code corrections, explanations, and verification; numerical results were computed from the supplied datasets.

If you run `python main.py 5`, it will load a dataset with two features and a very obvious clustering structure. It will then apply the  $k$ -means algorithm with a random initialization. The result of applying the algorithm will thus depend on the randomization, but a typical run might look like this:



(Note that the colours are arbitrary – this is the label switching issue.) But the “correct” clustering (that was used to make the data) is this:



## 5.1 Selecting Among $k$ -means Initializations [7 points]

If you run the demo several times, it will find different clusterings. To select among clusterings for a *fixed* value of  $k$ , one strategy is to minimize the sum of squared distances between examples  $x_i$  and their means  $w_{y_i}$ ,

$$f(w_1, w_2, \dots, w_k, y_1, y_2, \dots, y_n) = \sum_{i=1}^n \|x_i - w_{y_i}\|_2^2 = \sum_{i=1}^n \sum_{j=1}^d (x_{ij} - w_{y_i j})^2.$$

where  $y_i$  is the index of the closest mean to  $x_i$ . This is a natural criterion because the steps of  $k$ -means alternately optimize this objective function in terms of the  $w_c$  and the  $y_i$  values.

1. In the `kmeans.py` file, complete the `error()` method. `error()` takes as input the data used in fit (`X`), the indices of each examples' nearest mean (`y`), and the current value of means (`means`). It returns the value of this above objective function. [Submit this code](#). What trend do you observe if you print the value of this error after each iteration of the  $k$ -means algorithm? [4 points]

Answer:

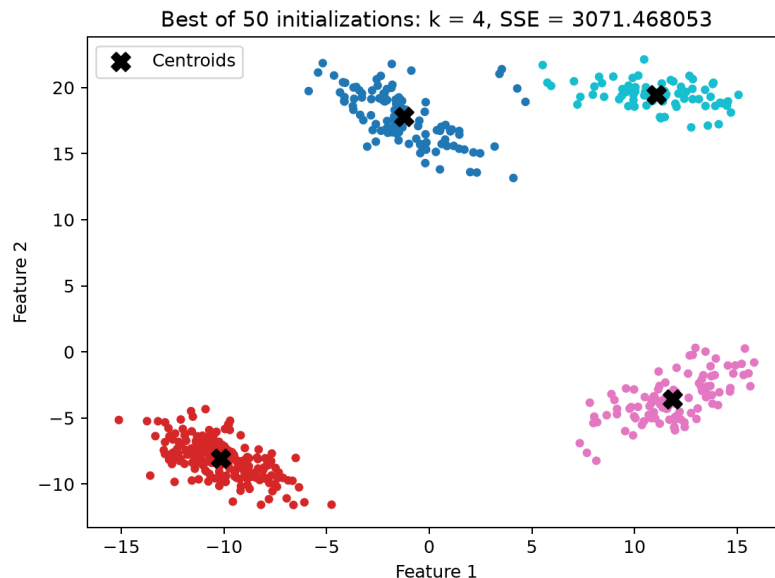
```
def error(self, X, y, means):
    return float(np.sum((X - means[np.asarray(y, dtype=int)])) ** 2)
```

Answer: The error is non-increasing after each assignment/update iteration and eventually becomes constant when the assignments stop changing. Assigning each point to its nearest mean and replacing each mean by its assigned points' average each minimize the objective with the other variables fixed.

2. Run  $k$ -means 50 times (with  $k = 4$ ) and take the one with the lowest error. Report the lowest error obtained. Visualize the clustering obtained by this model, and submit your plot. [3 points]

Answer: The lowest error obtained was 3071.468053.

Answer:



## 5.2 Selecting $k$ in $k$ -means [8 points]

We now turn to the task of choosing the number of clusters  $k$ .

1. Explain why we should not choose  $k$  by taking the value that minimizes the error value. [2 points]

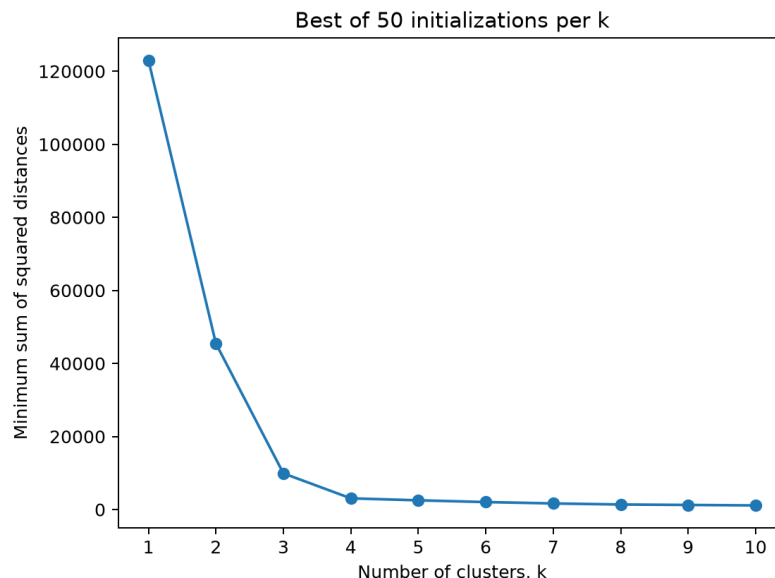
Answer: The globally minimized training objective is non-increasing as  $k$  increases, since additional centers can reproduce or improve a smaller model. At  $k = n$ , placing a center at every example gives zero error, so minimizing this criterion alone rewards complexity rather than a meaningful number of clusters.

2. Is evaluating the error function on validation (or test data) a suitable approach to choosing  $k$ ? [2 points]

Answer: Not by itself: more centers generally reduce nearest-center distances even for unseen points, so this distortion criterion does not adequately penalize complexity. Unlike supervised classification error, it need not expose overfitting as  $k$  grows; an elbow or an appropriate complexity penalty is needed.

3. Hand in a plot of the minimum error found across 50 random initializations, as a function of  $k$ , taking  $k$  from 1 to 10. [2 points]

Answer:



4. The *elbow method* for choosing  $k$  consists of looking at the above plot and visually trying to choose the  $k$  that makes the sharpest “elbow” (the biggest change in slope). What values of  $k$  might be reasonable according to this method? Note: there is not a single correct answer here; it is somewhat open to interpretation and there is a range of reasonable answers. [2 points]

Answer:  $k = 3$  and  $k = 4$  are reasonable elbow choices. The error drops steeply through  $k = 3$ , falls further from about 9909.52 to 3071.47 at  $k = 4$ , and improves much less beyond that. I would choose  $k = 4$ , which also captures the four visible groups.

## 6 Very-Short Answer Questions [18 points]

Additional reference for optimization bias: CPSC 340, Lecture 5, Mark Schmidt. *Assistance: OpenAI Codex assisted with code corrections, explanations, and verification; numerical results were computed from the supplied datasets.*

Write a short one or two sentence answer to each of the questions below. Make sure your answer is clear and concise.

1. What is a reason that the the data may not be IID in the email spam filtering example from lecture?

Answer: Emails from the same sender or spam campaign can be correlated, violating independence. The distribution can also change over time as topics and spam strategies change, violating identical distribution.

2. Why can't we (typically) use the training error to select a hyper-parameter?

Answer: Training error is optimistically biased because the model was fitted to those same observations. Minimizing it over hyperparameters tends to favor overly flexible models, so we use held-out validation or cross-validation instead.

3. What is the effect of the training or validation set size  $n$  on the optimization bias, assuming we use a parametric model?

Answer: For a fixed-complexity parametric model and a fixed amount of model selection, increasing the training or validation sample size generally reduces optimization bias: empirical errors fluctuate less, leaving less sampling noise to exploit. As  $n$  grows, the optimistically low error caused by selecting on that set tends to shrink.

4. What is an advantage and a disadvantage of using a large  $k$  value in  $k$ -fold cross-validation?

Answer: A larger  $k$  trains each model on a fraction  $(k - 1)/k$  of the data, making its training size closer to that of the final model and reducing this source of bias. It costs more computation because  $k$  models must be fitted and evaluated.

5. Recall that false positive in binary classification means  $\hat{y}_i = 1$  while  $\tilde{y}_i = 0$ . Give an example of when increasing false positives is an acceptable risk.

Answer: In screening for a serious disease, accepting more false positives can be worthwhile if it reduces missed cases. A positive screening result can be checked with a confirmatory test before treatment.

6. Why can we ignore  $p(x_i)$  when we use naive Bayes?

Answer: For a fixed input  $x_i$ ,  $p(x_i)$  is the same positive normalizing factor for every candidate class. It therefore does not change the class that maximizes the posterior probability.

7. For each of the three values below in a naive Bayes model, say whether it's better considered as a parameter or a hyper-parameter:

- (a) Our estimate of  $p(y_i)$  for some  $y_i$ .

Answer: parameter

- (b) Our estimate of  $p(x_{ij} | y_i)$  for some  $x_{ij}$  and  $y_i$ .

Answer: parameter

- (c) The value  $\beta$  in Laplace smoothing.

Answer: hyper-parameter

8. Both supervised learning and clustering models take in an input  $x_i$  and produce a label  $y_i$ . What is the key difference between these types of models?

Answer: Supervised learning learns to predict externally supplied target labels from labeled examples. Clustering receives no target labels and assigns arbitrary group identifiers based on similarities in the inputs.

9. In  $k$ -means clustering the clusters are guaranteed to be convex regions. Are the areas that are given the same label by kNN also convex?

Answer: No. A class region can be a non-convex or disconnected union of neighbourhoods; even for 1-NN, several separate Voronoi cells may share a class label while a different class lies between them.